

XEROX

XEROX

Printer thanks joe

Spruce version 11.2 -- spooler version 11.2

File: DSK: KMP; .XEH 1

Creation date: 22 DEC 1984 1934

For: KMP

5 total sheets = 4 pages, 1 copy.

XEROX

XEROX

```
;;: -*- Mode:LISP; -*-
```

```
(COMMENT ;For MacLisp debugging only
```

```
(DEFPROP DEFINE-OPERATION (LAMBDA (X) NIL) FEXPR)
(DEFMACRO DEFINE-SYNTAX ((NAME . BVL) . BODY)
  '(DEFMACRO ,NAME ,BVL ,@BODY))
(DEFMACRO LAMBDA (BVL &REST BODY)
  '#'(LAMBDA ,BVL ,@BODY))
(DEFPROP MAP MAPCAR EXPR)
(DEFUN GENERATE-SYMBOL (NAME) (SYMBOLCONC NAME '-17))
(DEFUN NULL? (X) (NULL X))
(DEFVAR ELSE T)
(DEFPROP BEGIN PROGN EXPR)
```

```
)
```

```
;;; (DEFINE-CONDITION name locals [parents])
```

```
(DEFINE-SYNTAX (DEFINE-CONDITION NAME LOCALS . REST)
  (LET ((PARENTS (IF (NULL? REST) '() (CAR REST))))
    '(BEGIN (DEFINE ,NAME (CONDITION ,NAME ,LOCALS ,PARENTS))
      ,@(MAP (LAMBDA (LOCAL)
        '(DEFINE-OPERATION (,(SYMBOLCONC NAME '- (CAR LOCAL))
          ,NAME)))
        LOCALS))))
```

```
(DEFINE-SYNTAX (CONDITION NAME LOCALS PARENTS)
  (LET ((ARGS-VAR (GENERATE-SYMBOL 'ARGS))
        (SELF-VAR (GENERATE-SYMBOL 'SELF)))
    '(OBJECT (LAMBDA (,ARGS-VAR)
      ,(LET ((BODY '(LET ,(MAP (LAMBDA (LOCAL)
        '(',(CAR LOCAL)
          (COND ((MEMQ ',(CAR LOCAL)
            ,ARGS-VAR)
              => CADR)
            (ELSE ,(CADR LOCAL))))))
        LOCALS)
        (OBJECT ()
          ,@(MAP (LAMBDA (LOCAL)
            '(',(SYMBOLCONC NAME '- (CAR LOCAL))
              ,(CAR LOCAL)))
            LOCALS)
            (=> CONDITION-HANDLER))))))
      (IF (NULL? PARENTS) BODY
        '(JOIN ,BODY
          ,@(MAP (LAMBDA (PARENT)
            '(',PARENT ,ARGS-VAR)
              PARENTS))))))
    ((IDENTIFICATION SELF) ',NAME))))
```

```
;;; Note:
```

```
;;; Think about: (IF (NOT (GET ... KEY)) (PUT ... KEY default))
;;; instead. Could save storage, avoid useless computation, simplify
;;; message passing.
```

```
;;; (CONDITION-BIND ((type1 handler1)
;;;                  (type2 handler2)
;;;                  ...)
```

```
;;;
```

```
;;; Handlers must be functions of three args:
```

```
;;; the condition
;;; a return continuation
;;; a dismiss continuation
```

```
;;; Note well:
```

```
;;; This will not allow DISMISS to work.
;;; Need to think up some mechanism for calling continuation.
;;; Gonna need to make a protocol for pushing condition handlers in
;;; the condition-bind rather than setting them.
```

```

(DEFINE-SYNTAX (CONDITION-BIND DATA . BODY)
  (COND ((NULL? DATA)
    (COND ((NULL? BODY) 'NIL)
      ((NULL? (CDR BODY)) (CAR BODY))
      (ELSE '(BEGIN ,@BODY))))
    (ELSE
      '(BIND (((CONDITION-HANDLER ,(CAAR DATA)) ,(CADAR DATA)))
        (CONDITION-BIND ,(CDR DATA) ,@BODY))))))

```

```

(DEFINE-SYNTAX (CONDITION-LET DATA . BODY)
  (COND ((NULL? DATA)
    (COND ((NULL? BODY) 'NIL)
      ((NULL? (CDR BODY)) (CAR BODY))
      (ELSE '(BEGIN ,@BODY))))
    (ELSE
      '(LET ((,(CAAR DATA)
        (LET ((STORAGE ,(CADAR DATA)))
          (OBJECT NIL
            ((CONDITION? SELF) T)
            ((CONDITION-HANDLER SELF) STORAGE)
            (((SETTER CONDITION-HANDLER) SELF FN)
              (SET STORAGE FN))))))
        (CONDITION-LET ,(CDR DATA) ,@BODY))))))

```

```

;;; (SIGNAL condition . args)

```

```

(DEFINE (SIGNAL CONDITION PROCEED-CASES . ARGS)
  ((OR (CONDITION-HANDLER CONDITION) FALSE)
    ...))

```

```

(DEFINE (ERROR CONDITION PROCEED-CASES . ARGS)
  ((OR (CONDITION-HANDLER CONDITION) DEBUGGER)
    ...))

```

```

;;; (DEFINE-PROCEED-TYPE type ...)
;;; (PROCEED-CASE ...)

```

;;; Definitions not in use

(COMMENT ;This assumes (FLUID-CONDITION-HANDLER-STATE) returns an alist

(LET ((HANDLER-STATE '()))

(DEFINE FLUID-CONDITION-HANDLER-STATE (LOCATIVE HANDLER-STATE)))

(DEFINE-SYNTAX (CONDITION-BIND DATA . BODY)

(COND ((NULL? DATA)

(COND ((NULL? BODY) 'NIL)

((NULL? (CDR BODY)) (CAR BODY))

(ELSE '(BEGIN ,@BODY))))

(T

'(BIND (((FLUID-CONDITION-HANDLER-STATE)

(APPEND (LIST ,@(MAP (LAMBDA (DATUM) '(LIST ,@DATUM))

DATA))

(FLUID-CONDITION-HANDLER-STATE))))

,@BODY))))

(DEFINE (INVOKE-CONDITION-HANDLERS CONDITION-OBJECT FAILURE-ACTION)

(CATCH RETURN

(DO ((H (FLUID-CONDITION-HANDLER-STATE) (CDR H)))

((NULL? H) (FAILURE-ACTION CONDITION-OBJECT))

(CATCH DECLINE

(LET ((CLAUSE (CAR H)))

(IF ((CAR CLAUSE) CONDITION-OBJECT)

((CADR CLAUSE) CONDITION-OBJECT

RETURN

(LAMBDA () (DECLINE T))))))

)


```
;;; Local Modes:  
;;; Lisp BIND Indent:1  
;;; Lisp CATCH Indent:1  
;;; Lisp OBJECT Indent:1  
;;; Lisp CONDITION-BIND Indent:1  
;;; End:
```